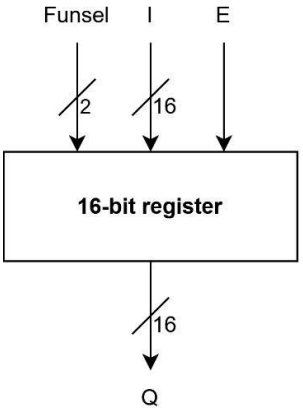
BLG222E Computer Organization

Project 1

Due Date: 02.04.2026 23:59

In this project, registers, register files, and memory units will be designed and implemented. It has 4 parts. Design each part **as a module**, so that it can be reused in other parts. **You must simulate each module for each combination of input.**

(Part-1) Design a **16-bit register**. This register has 4 functionalities that are controlled by 2-bit control signals **(FunSel)** and an enable input **(E).** The graphic symbol of the registers and the characteristic table are shown in Figure 1. Symbol **ϕ** means **"don't care"**. This register will be used in later parts.



E	FunSel	Q^+
0	**ϕ**	Q (Retain value)
1	00	0 (Clear)
1	01	I (Load)
1	10	Q+1 (Increment)
1	11	Q-1 (Decrement)

Figure 1: Graphic symbol of a 16-bit register and the characteristic table, respectively.

(Part-2) Design register files (a structure that contains many registers) that work as follows.

(Part-2a) Design the **Register File (RF)** shown in Figure 2, which consists of four 16-bit general-purpose registers: **R1**, **R2**, **R3**, and **R4**, and four 16-bit scratch registers: **S1**, **S2**, **S3**, and **S4.** The details of inputs and outputs are as follows.

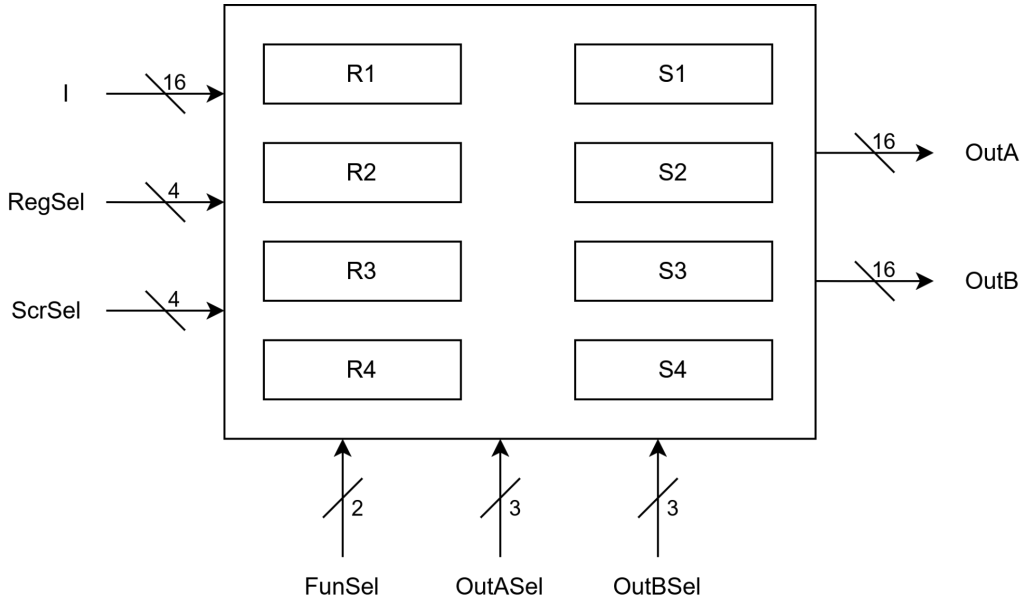


Figure 2: 16-bit general-purpose and scratch registers, inputs, and outputs.

OutASel and **OutBSel** are used to feed the output lines **OutA** and **OutB**, respectively. 16 bits of selected registers are output to **OutA** and **OutB**. Table 1 shows the selection of output registers based on the **OutASel** and **OutBSel** control inputs.

Table 1: OutASel and OutBSel controls.

OutASel	OutA
000	R1
001	R2
010	R3
011	R4
100	S1
101	S2
110	S3
111	S4

OutBSel	OutB
000	R1
001	R2
010	R3
011	R4
100	S1
101	S2
110	S3
111	S4

RegSel and **ScrSel** are 4-bit signals that select the registers to apply the function that is determined by the 2-bit **FunSel** signal, which is given in Table 2. The selected registers by **RegSel** and **ScrSel** are shown in Tables 3 and 4, respectively.

Table 2: FunSel Control Input.

FunSel	R_x^+ (Next State)
00	0 (Clear)
01	I (Load)
10	R_x +1 (Increment)
11	R_x-1 (Decrement)

Table 3: RegSel Control Input

RegSel	Enable General Purpose Registers	RegSel	Enable General Purpose Registers
0000	All general purpose registers are enabled. (Function selected by FunSel will be applied to R1, R2, R3 and R4.)	1000	R2, R3, and R4 are enabled. (Function selected by FunSel will be applied to R2, R3, and R4.)
0001	R1, R2 and R3 are enabled. (Function selected by FunSel will be applied to R1, R2, and R3.)	1001	R2 and R3 are enabled. (Function selected by FunSel will be applied to R2 and R3.)
0010	R1, R2, and R4 are enabled. (Function selected by FunSel will be applied to R1, R2, and R4.)	1010	R2 and R4 are enabled. (Function selected by FunSel will be applied to R2 and R4.)
0011	R1 and R2 are enabled. (Function selected by FunSel will be applied to R1 and R2.)	1011	Only R2 is enabled. (Function selected by FunSel will be applied to R2.)
0100	R1, R3, and R4 are enabled. (Function selected by FunSel will be applied to R1, R3, and R4.)	1100	R3 and R4 are enabled. (Function selected by FunSel will be applied to R3 and R4.)
0101	R1 and R3 are enabled. (Function selected by FunSel will be applied to R1 and R3.)	1101	Only R3 is enabled. (Function selected by FunSel will be applied to R3.)
0110	R1 and R4 are enabled. (Function selected by FunSel will be applied to R1 and R4.)	1110	Only R4 is enabled. (Function selected by FunSel will be applied to R4.)
0111	Only R1 is enabled. (Function selected by FunSel will be applied to R1.)	1111	NO general purpose register is enabled. (All registers retain their values.)

Table 4: ScrSel Control Input

ScrSel	Enable General Purpose Registers	ScrSel	Enable General Purpose Registers
0000	All general purpose registers are enabled. (Function selected by FunSel will be applied to S1, S2, S3, and S4.)	1000	S2, S3, and S4 are enabled. (Function selected by FunSel will be applied to S2, S3, and S4.)
0001	S1, S2, and S3 are enabled. (Function selected by FunSel will be applied to S1, S2, and S3.)	1001	S2 and S3 are enabled. (Function selected by FunSel will be applied to S2 and S3.)
0010	S1, S2, and S4 are enabled. (Function selected by FunSel will be applied to S1, S2, and S4.)	1010	S2 and S4 are enabled. (Function selected by FunSel will be applied to S2 and S4.)
0011	S1 and S2 are enabled. (Function selected by FunSel will be applied to S1 and S2.)	1011	Only S2 is enabled. (Function selected by FunSel will be applied to S2.)
0100	S1, S3, and S4 are enabled. (Function selected by FunSel will be applied to S1, S3, and S4.)	1100	S3 and S4 are enabled. (Function selected by FunSel will be applied to S3 and S4.)
0101	S1 and S3 are enabled. (Function selected by FunSel will be applied to S1 and S3.)	1101	Only S3 is enabled. (Function selected by FunSel will be applied to S3.)
0110	S1 and S4 are enabled. (Function selected by FunSel will be applied to S1 and S4.)	1110	Only S4 is enabled. (Function selected by FunSel will be applied to S4.)
0111	Only S1 is enabled. (Function selected by FunSel will be applied to S1.)	1111	NO general purpose register is enabled. (All registers retain their values.)

(Part-2b) Design the **Address Register File (ARF)** system shown in Figure 3, which consists of three 16-bit address registers: **Program Counter (PC)**, **Address Register (AR)**, **and Stack Pointer (SP)**.

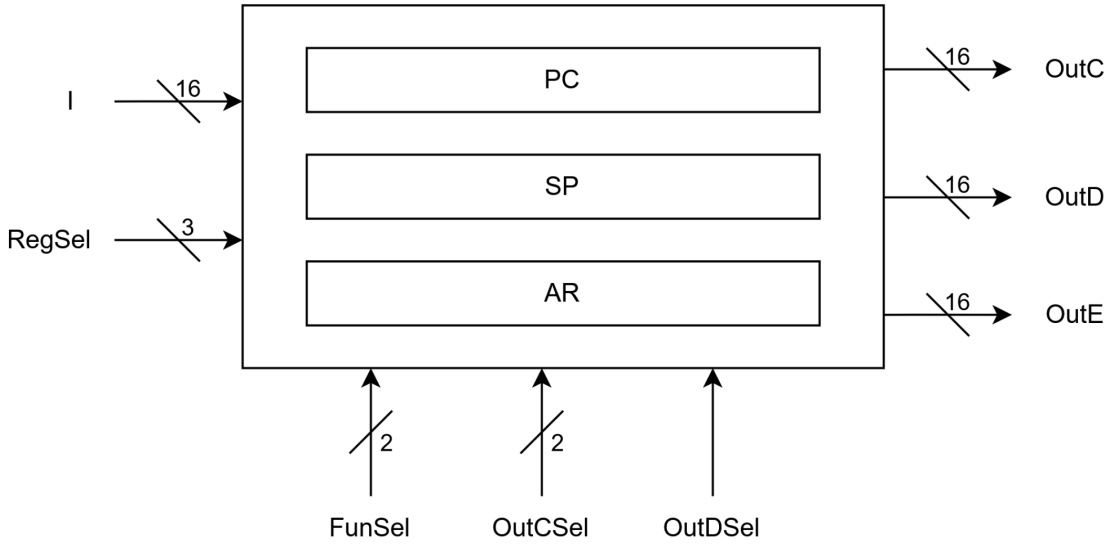


Figure 3: 16-bit address registers, inputs, and outputs.

OutCSel and **OutDSel** are used to feed output lines **OutC** and **OutD**, respectively. 16 bits of the selected registers are output to **OutC** and **OutD**. Table 5 shows the selection of output registers based on the **OutCSel** and **OutDSel** control inputs. **OutE** always shows the PC register.

Table 5: OutCSel and OutDSel controls.

OutCSel	OutC
00	PC
01	PC
10	AR
11	SP

OutDSel	OutD
0	AR
1	SP

RegSel is a 3-bit signal that selects the registers to apply the function that is determined by the 2-bit **FunSel** signal, which is given in Table 6. The selected registers by **RegSel** are shown in Table 7.

Table 6: FunSel Control Input.

FunSel	R_x^+ (Next State)
00	0 (Clear)
01	I (Load)
10	R_x +1 (Increment)
11	R_x-1 (Decrement)

Table 7: RegSel Control Input

RegSel	Enable General Purpose Registers	RegSel	Enable General Purpose Registers
000	All registers are enabled. (Function selected by FunSel will be applied to PC, SP, and AR.)	100	SP and AR are enabled. (Function selected by FunSel will be applied to SP and AR.)
001	PC and SP are enabled. (Function selected by FunSel will be applied to PC and SP.)	101	Only SP is enabled. (Function selected by FunSel will be applied to SP.)
010	PC and AR are enabled. (Function selected by FunSel will be applied to PC.)	110	Only AR is enabled. (Function selected by FunSel will be applied to AR.)
011	SP and AR are enabled. (Function selected by FunSel will be applied to SP and AR.)	111	NO registers are enabled. (All registers retain their values.)

(Part-3) Design memory units (a structure that contains memory and a handler register) that work as follows.

(Part-3a) Design a **Data Memory Unit (DMU)** that contains **Random Access Memory (RAM)** and a **Data Register (DR)** to handle data read and write into memory.

RAM is provided to you in the Memory.v file. A block diagram of RAM is given in Figure 4. It takes a 16-bit address and 8-bit input data. When WR is 0, 8-bit data can be read from memory; when WR is 1, 8-bit data can be written to memory. The CS signal is used as an enable signal (CS=0 enable).

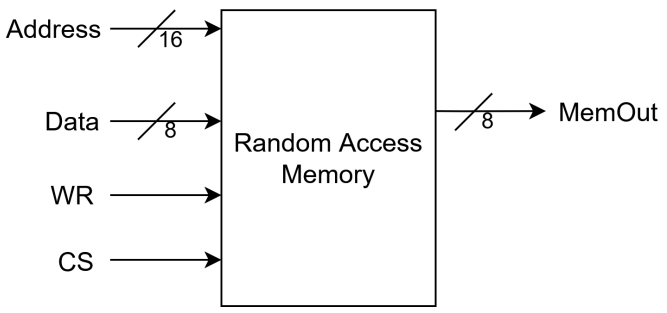
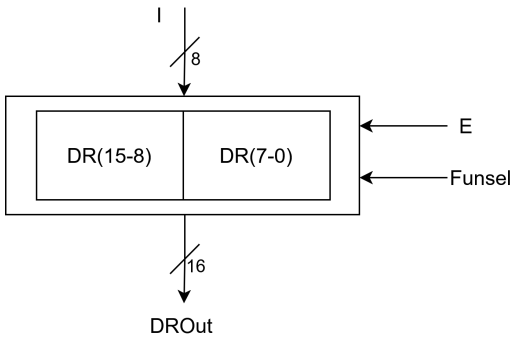


Figure 4: Graphic symbol of RAM register.

Design a 16-bit **Data Register (DR)** register whose block diagram and function table are given in Figure 5. This register can store 16-bit binary data. The input data bus of the DR register file, similar to the IR register, is limited to 8 bits. Therefore, only 8 bits of data can be loaded into the register per clock cycle, requiring two cycles to fully load 16 bits of data.



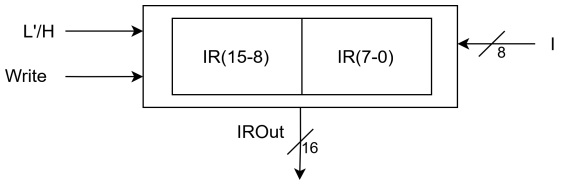
E	FunSel	DR⁺
0	ϕ	DR (Retain value)
1	0	DR (7-0) ← I (Load LSB)
1	1	DR (15-8) ← I (Load MSB)

Figure 5: Graphic symbol of the DR register and its characteristic table, respectively.

Combine the RAM and DR into the Data Memory Unit (DMU). This unit is controlled by a CS, WR, and a FunSel inputs to read/write memory. The CS signal serves as the enable signal (CS=1 enable). You can directly connect the DMU WR input to the RAM WR input. FunSel is used to control DR FunSel.

(Part-3b) Design an **Instruction Memory Unit (IMU)** that contains **Read Only Memory (ROM)** and **Instruction Register (IR)**.

Block diagram and function table of the **Instruction Register (IR)** are given in Figure 6. This register can store 16-bit binary data. However, the input of this register is only 8 bits. Therefore, using the 8-bit input bus, you can load either the lower (bits 7-0) or higher (bits 15-8) half. This decision is made by the **L'H** signal.



L'H	Write	IR⁺
ϕ	0	IR (Retain value)
0	1	IR (7-0) ← I (Load LSB)
1	1	IR (15-8) ← I (Load MSB)

Figure 6: Graphic symbol of the IR register and its characteristic table, respectively.

Design a ROM that is responsible for storing the program instructions. IR reads instructions from this memory. You should examine the provided test bench and RAM example to determine its structure.

Combine the ROM and IR into the Instruction Memory Unit (IMU). This unit is controlled by a CS signal and an L'H input to read from memory. The CS signal serves as the enable signal (CS=1 enable). The unit provides two outputs: IROut and IMUOut. IROut directly outputs the full 16-bit value of the IR. IMUOut is a 16-bit output that displays only the 8 LSB of the IR, with the 8 MSB padded with zeros.

(Part-4) Design an Arithmetic Logic Unit (ALU) that has two 16-bit inputs, a 16-bit output, and a 4-bit output for **z**ero, **n**egative, **c**arry, and **o**verflow flags. The ALU is shown in Figure 7. The ALU functions and the flags that will be updated (i.e., **-** means that the flag will not be affected, and **+** means that the flag changes based on the ALUOut) are given in Table 8 when **WF** (Write Flag) is 1.

- **FunSel** selects the function of the ALU.
- **ALUOut** shows the result of the operation selected by FunSel and applied to the A and/or B inputs.
- **Arithmetic operations** are done using **2's complement** logic.
- **Z (zero)** bit is set if **ALUOut** is zero (e.g. when **NOT B** is zero).
- **C (carry)** bit is set if **ALUOut** sets the carry (e.g. when **LSL A** produces carry)
- **N (negative)** bit is set if the ALU operation generates a negative result (e.g. when **A-B** results in a negative number).
- **O (overflow)** bit is set if an overflow occurs (e.g. when **A+B** results in an overflow).
- Note that **Z|C|N|O** flags are stored in a **register**! The **Z** flag is the **MSB** of the register, and the **O** flag is the **LSB** of the register.

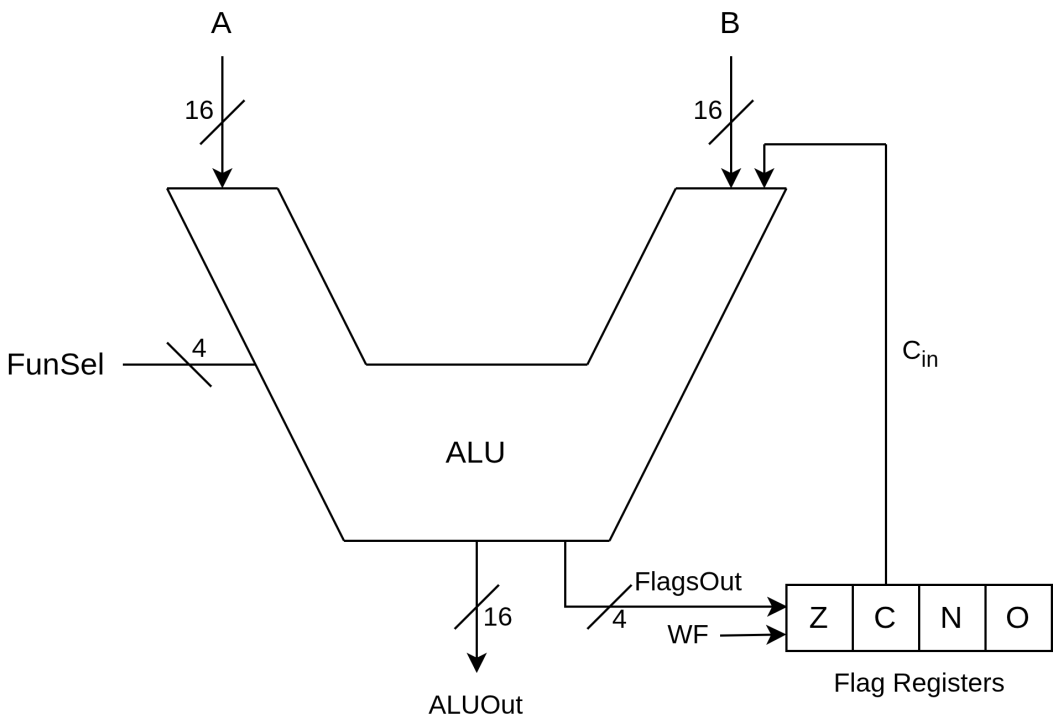


Figure 7: Arithmetic Logic Unit graphic.

Table 8: Characteristic table of ALU.

FunSel	ALUOut	Z	C	N	O
0000	A	+	-	+	-
0001	B	+	-	+	-
0010	NOT A	+	-	+	-
0011	NOT B	+	-	+	-
0100	A + B	+	+	+	+
0101	A + B + Carry	+	+	+	+
0110	A – B	+	+	+	+
0111	A AND B	+	-	+	-
1000	A OR B	+	-	+	-
1001	A XOR B	+	-	+	-
1010	A NAND B	+	-	+	-
1011	LSL A	+	+	+	-
1100	LSR A	+	+	+	-
1101	ASR A	+	-	-	-
1110	CSL A	+	+	+	-
1111	CSR A	+	+	+	-

(**C**ircular | **A**rithmetic | **L**ogical) Shift (**L**eft | **R**ight) operations are depicted in Figures 8, 9, and 10.

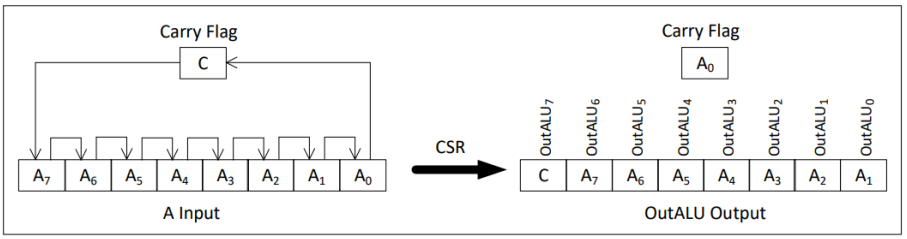


Figure 8: Circular shift operation.

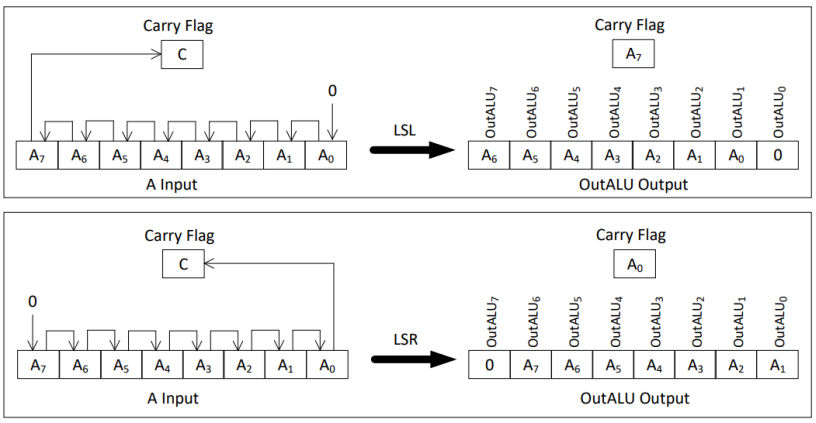


Figure 9: Logical shift operation.

Figure 10: Arithmetic shift operation.

(Part-4) Implement the organization in Figure 11. Please note that **the whole system uses the same single clock.**

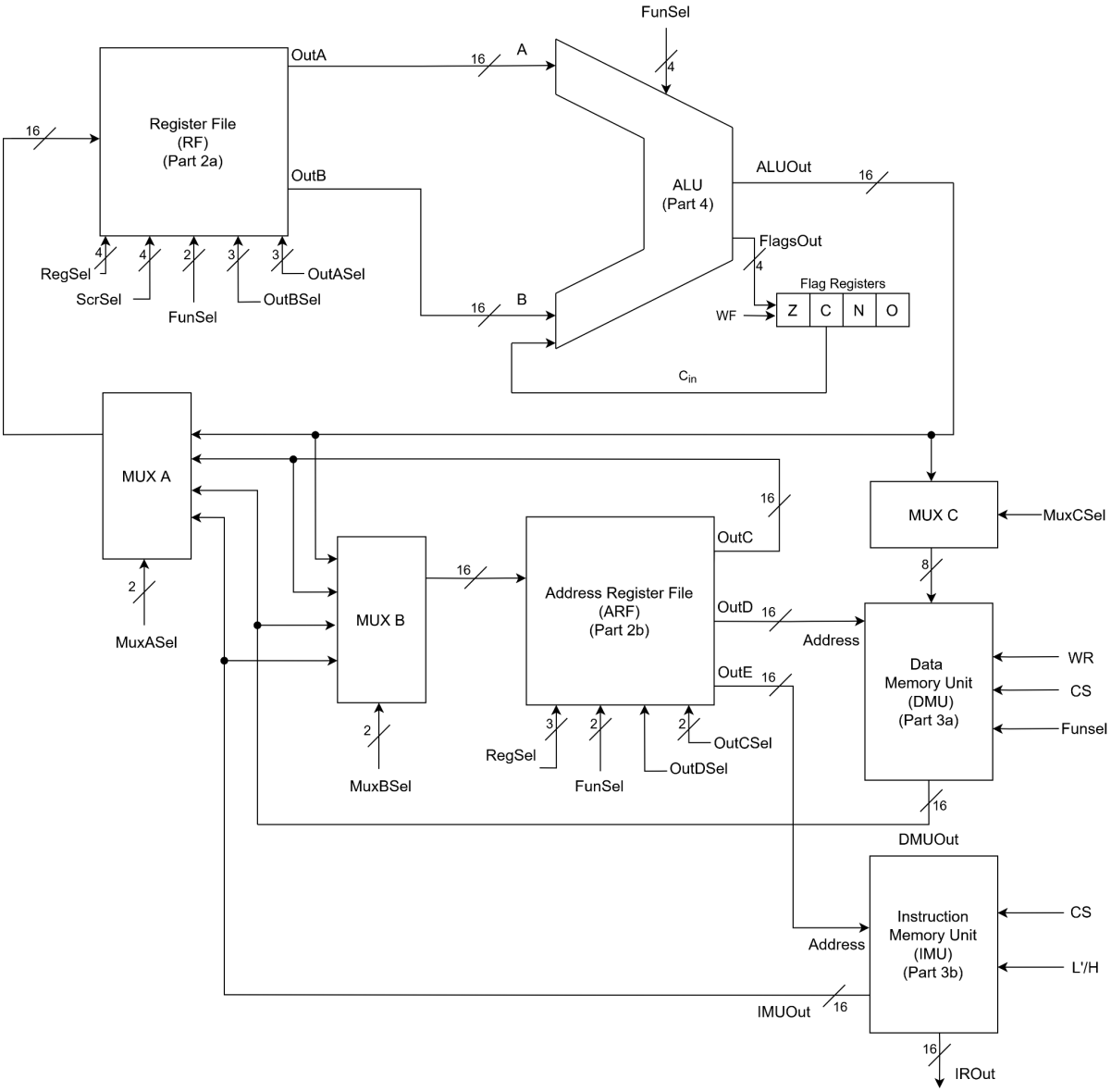


Figure 11: ALU System.

Table 9: Multiplexers of ALU Systems.

MuxASel	MuxAOut
00	ALUOut
01	OutC
10	DMUOut
11	IMUOut

MuxBSel	MuxAOut
00	ALUOut
01	OutC
10	DMUOut
11	IMUOut

MuxCSel	MUXCOut
0	ALUOut (7-0)
1	ALUOut (15-8)

Submission:

Implement your design in Verilog HDL, and upload a single compressed (zip) file to Ninova before the deadline. **Only one student** from each group should submit the project file (select one member of the group as the group representative for this purpose). Example submission file and module name declarations are given as project attachments. It is mandatory to fill out GroupMembers.xlsx (don't change its format). The compressed file should contain your modules file (.v) for each part, the given simulation file (.v), and a report that contains:

- the number&names of the students in the group
- list of control inputs and corresponding functions for your design
- explanations for each constructed part
- task distribution of each group member

Group work is expected for this project. All members of the group must design together. You must ensure that all modules work properly with the given simulation files. You are expected to get results by running the given .bat file.

The project will be evaluated only using simulation files by running Run.bat file on Vivado 2017.4. There will not be any partial grading for the designs. If your codes get any errors, you will not get any grades for that part, so make sure that your codes are working with the given simulation files. For detailed information about testing and submission procedures, check the provided *Running Simulation Tests.pdf* file.

There will not be any demonstration sessions. You may ask your questions on the Message Board; questions sent via email will not be answered.